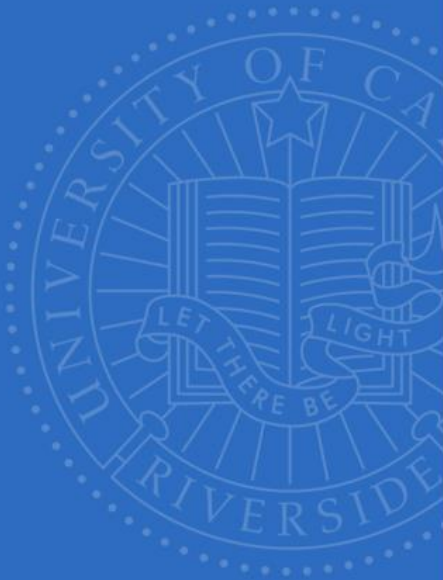


UCR



Memory Performance and Memory Coalescing

UNIVERSITY OF CALIFORNIA, RIVERSIDE

MEMORY ACCESS PERFORMANCE

Objective

- To learn that memory bandwidth is a first-order performance factor in a massively parallel processor
 - DRAM bursts, banks, and channels
 - All concepts are also applicable to modern multicore processors

Global Memory (DRAM) Bandwidth

Ideal

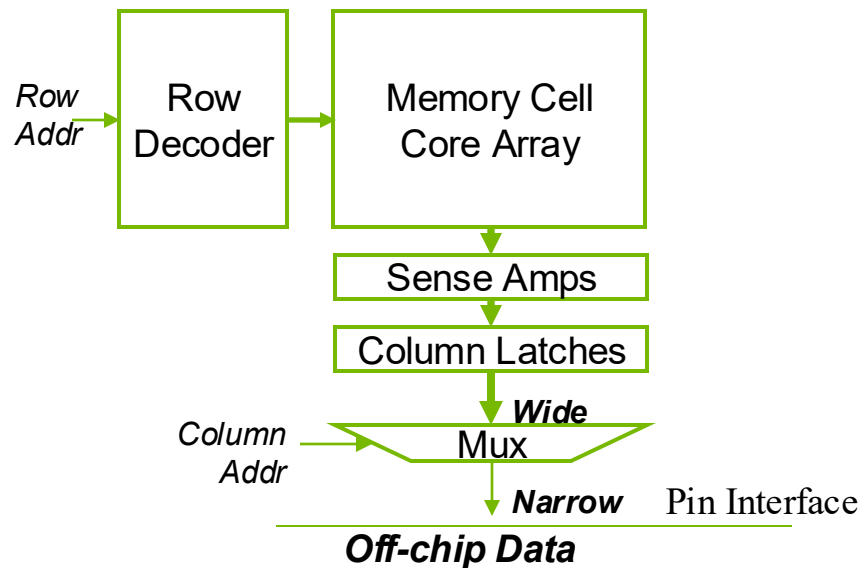


Reality

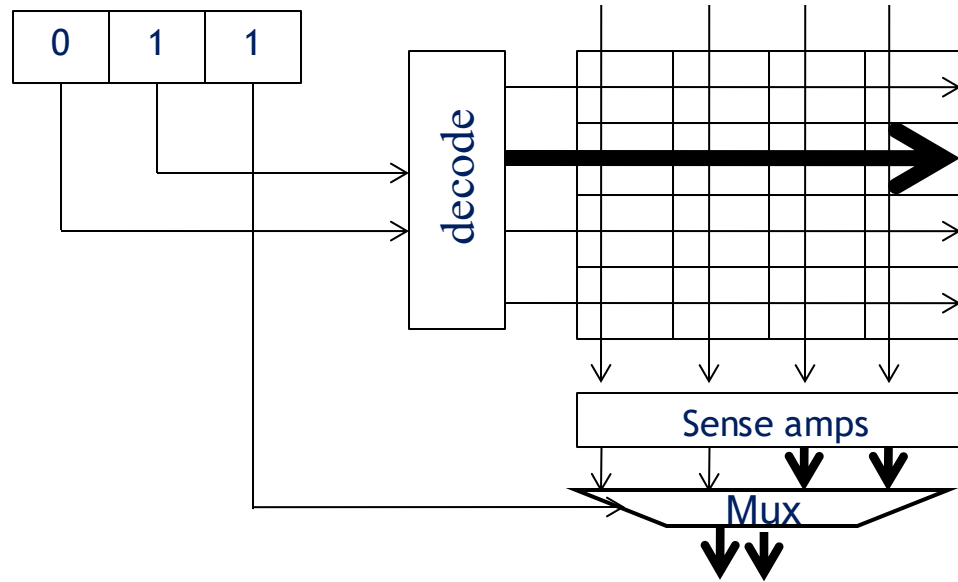


DRAM Core Array Organization

- Each DRAM core array has about 16M bits
- Each bit is stored in a tiny capacitor made of one transistor

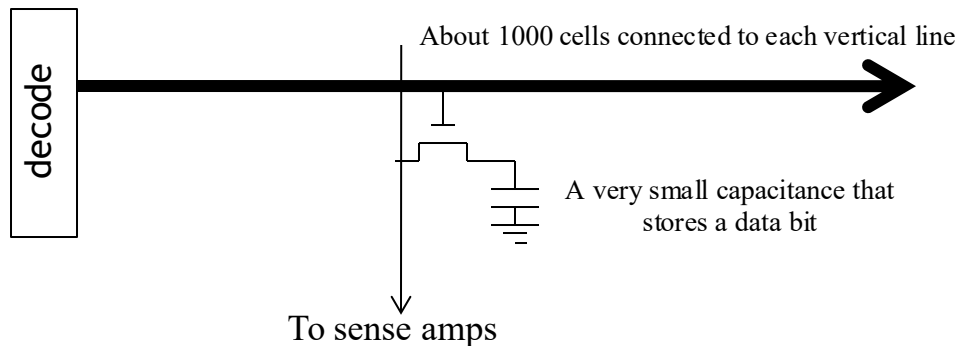


A very small (8x2-bit) DRAM Core Array



DRAM Core Arrays are Slow

- Reading from a cell in the core array is a very slow process
 - DDR: Core speed = $\frac{1}{2}$ interface speed
 - DDR2/GDDR3: Core speed = $\frac{1}{4}$ interface speed
 - DDR3/GDDR4: Core speed = $\frac{1}{8}$ interface speed
 - ... likely to be worse in the future



DRAM Bursting

- For DDR{2,3} SDRAM cores clocked at $1/N$ speed of the interface:
- Load ($N \times$ interface width) of DRAM bits from the same row at once to an internal buffer, then transfer in N steps at interface speed
 - DDR3/GDDR4: buffer width = $8X$ interface width

DRAM Bank Operation

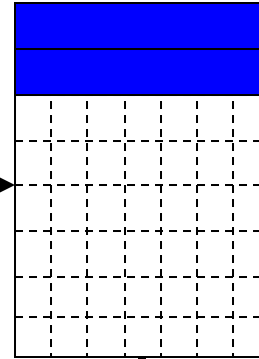
Access Address:
(Row 0, Column 0)
(Row 0, Column 1)
(Row 0, Column 85)
(Row 1, Column 0)

Row address 0

Row decoder

Columns

Rows



Row Buffer **CONFLICT !**

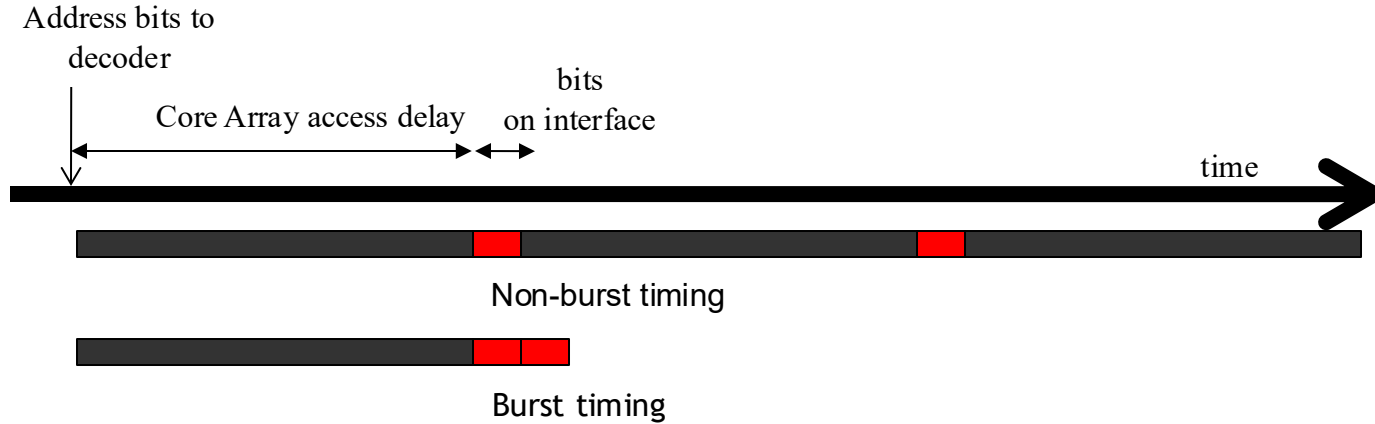
Column address 05

Column mux

Data

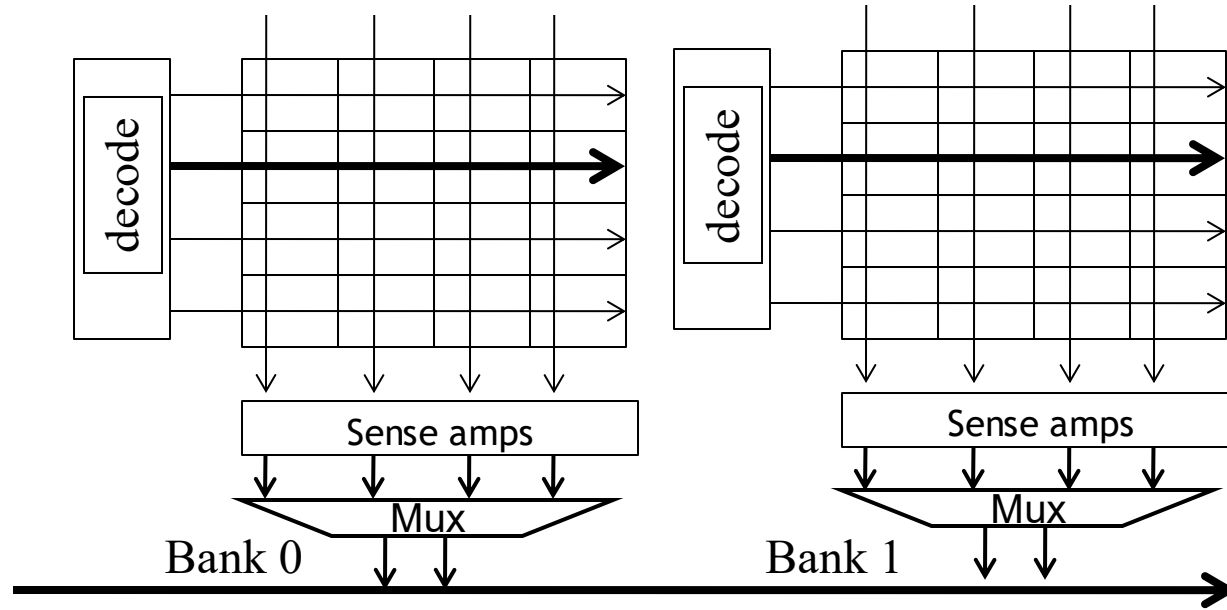


DRAM Bursting Timing Example



Modern DRAM systems are designed to always be accessed in burst mode. Burst bytes are transferred to the processor but discarded when accesses are not to sequential locations.

Multiple DRAM Banks



DRAM Bursting with Banking



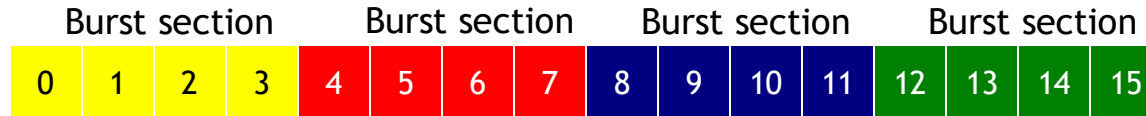
Single-Bank burst timing, dead time on interface



Multi-Bank burst timing, reduced dead time

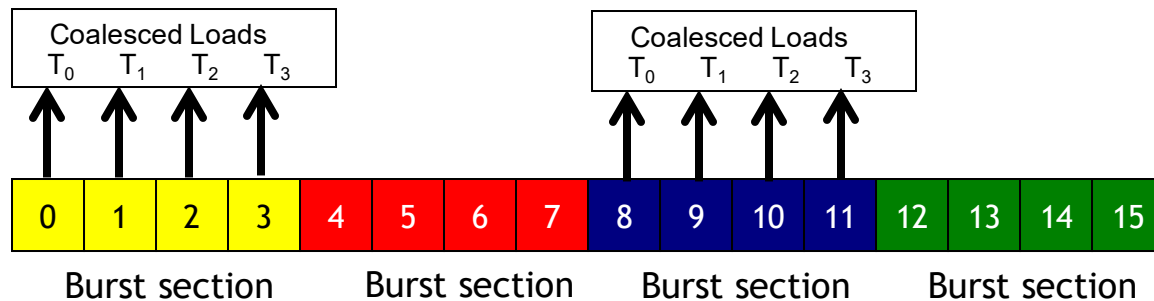
MEMORY COALESCING

DRAM Burst – A System View



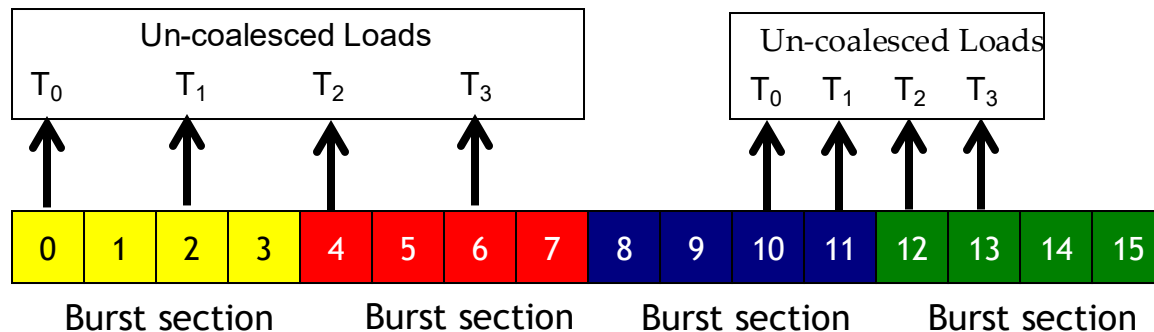
- Each address space is partitioned into burst sections
 - Whenever a location is accessed, all other locations in the same section are also delivered to the processor
- Basic example: a 16-byte address space, 4-byte burst sections
 - In practice, we have at least 4GB address space, burst section sizes of 128-bytes or more

Memory Coalescing



- When *all threads of a warp* execute a load instruction, if all accessed locations fall into the *same burst section*, *only one DRAM request* will be made and the access is fully coalesced.

Un-coalesced Accesses

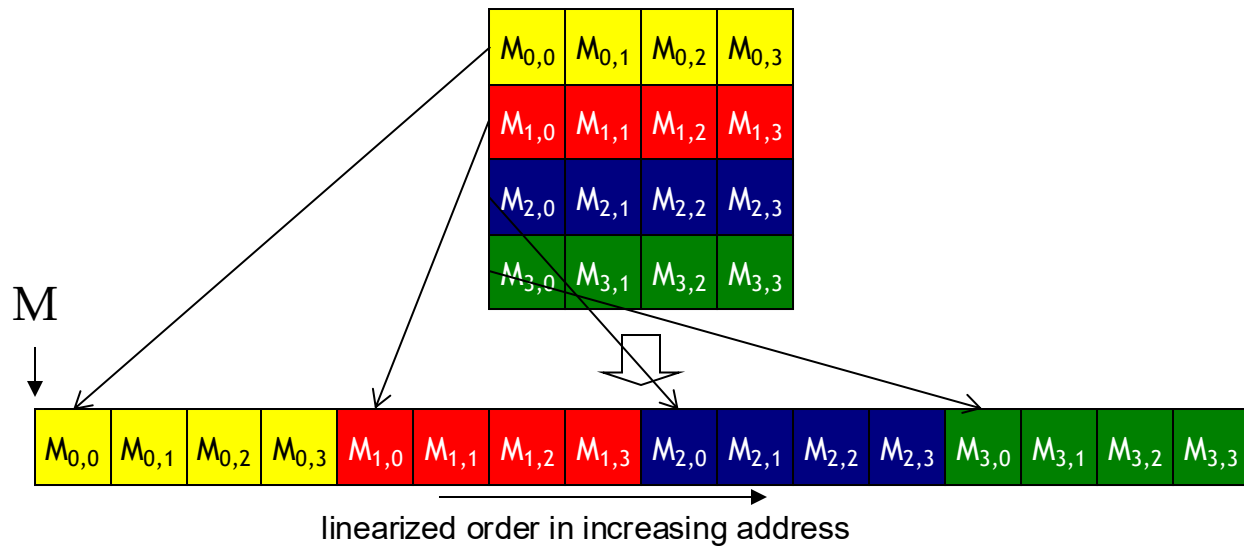


- When the accessed locations spread across burst section boundaries:
 - Coalescing fails
 - Multiple DRAM requests are made
 - The access is not fully coalesced.
- Some of the bytes accessed and transferred are not used by the threads

How to judge if an access is coalesced?

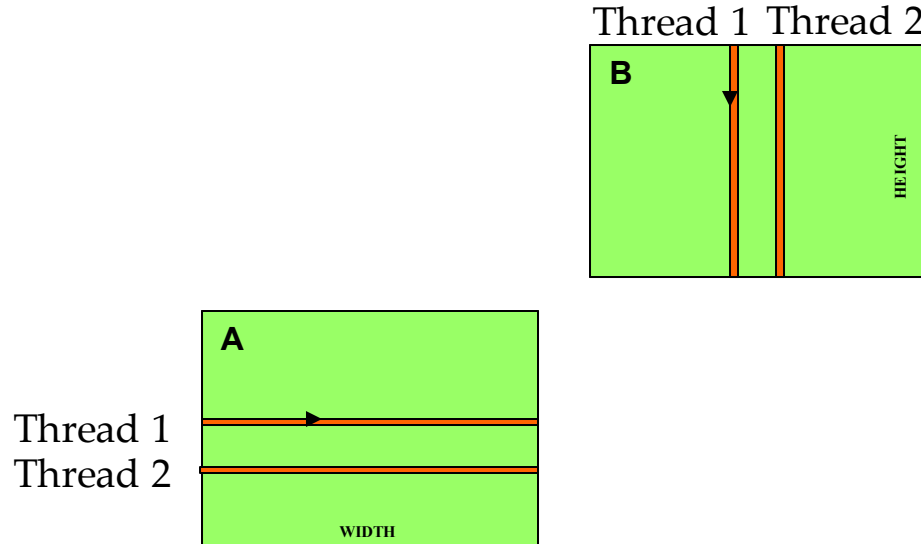
- Accesses **in a warp** are to **consecutive locations** if the index in an array access is in the form of
 - $A[(\text{expression with terms independent of threadIdx.x}) + \text{threadIdx.x}];$

A 2D C Array in Linear Memory Space

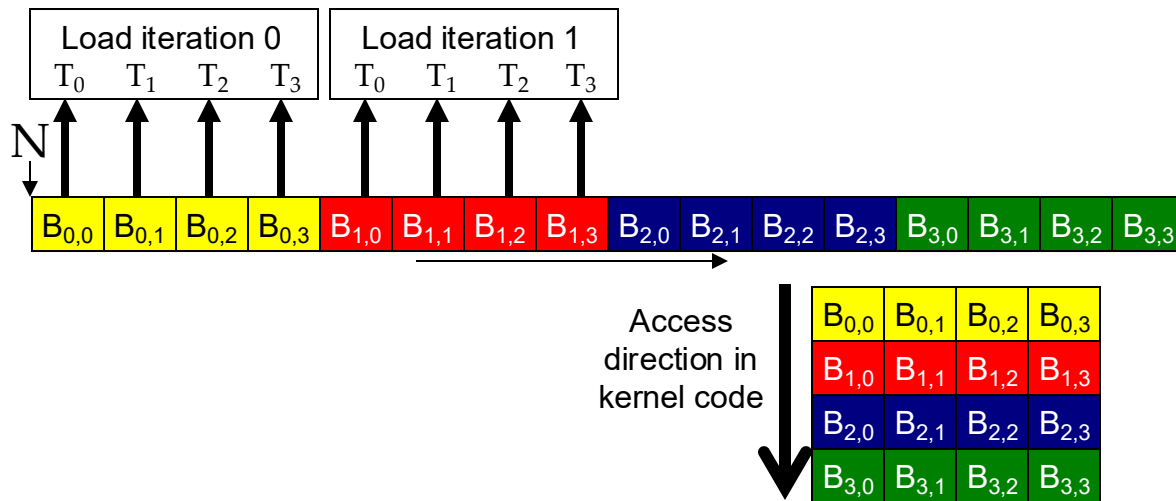


Two Access Patterns of Basic Matrix Multiplication

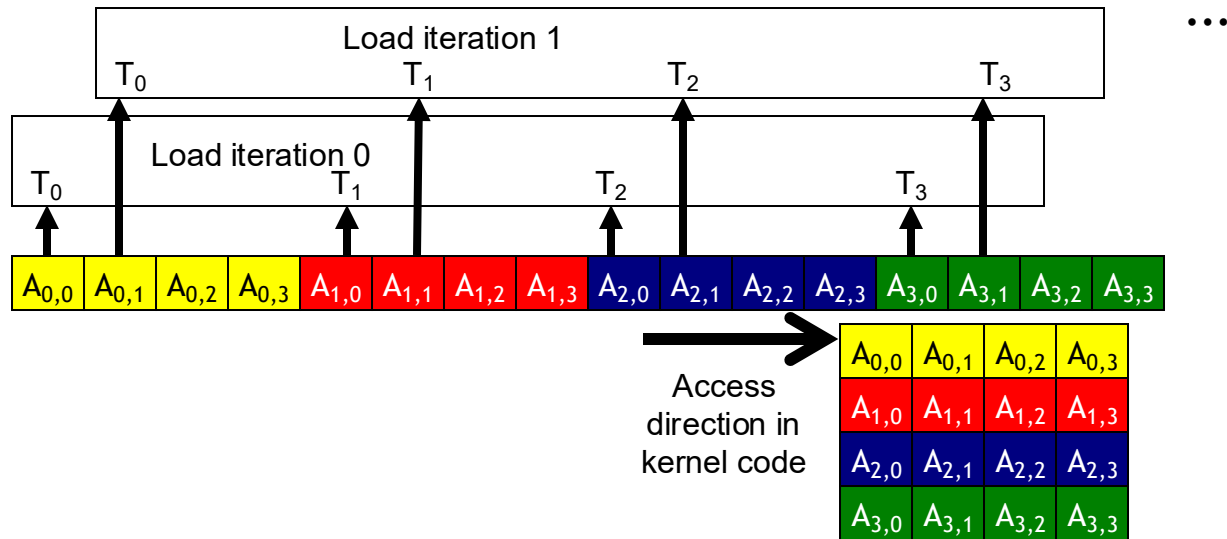
Let's assume the following thread mapping
(*not representative of our actual implementation*)



B accesses are coalesced



A Accesses are Not Coalesced



A Basic Matrix Multiplication

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int Width) {  
  
    // Calculate the row index of the P element and M  
    int Row = blockIdx.y*blockDim.y+threadIdx.y;  
  
    // Calculate the column index of P and N  
    int Col = blockIdx.x*blockDim.x+threadIdx.x;  
  
    if ((Row < Width) && (Col < Width)) {  
        float Pvalue = 0;  
        // each thread computes one element of the block sub-matrix  
        for (int k = 0; k < Width; ++k) {  
            Pvalue += M[Row*Width+k]*N[k*Width+Col];  
        }  
        P[Row*Width+Col] = Pvalue;  
    }  
}
```

Tiled Matrix Multiply access use 1D indexing

```
int Row = by * blockDim.y + ty;
int Col =  bx * blockDim.x + tx;
```

→ $M[\text{Row}][p * \text{TILE_WIDTH} + tx]$
 $M[\text{Row} * \text{Width} + p * \text{TILE_WIDTH} + tx]$

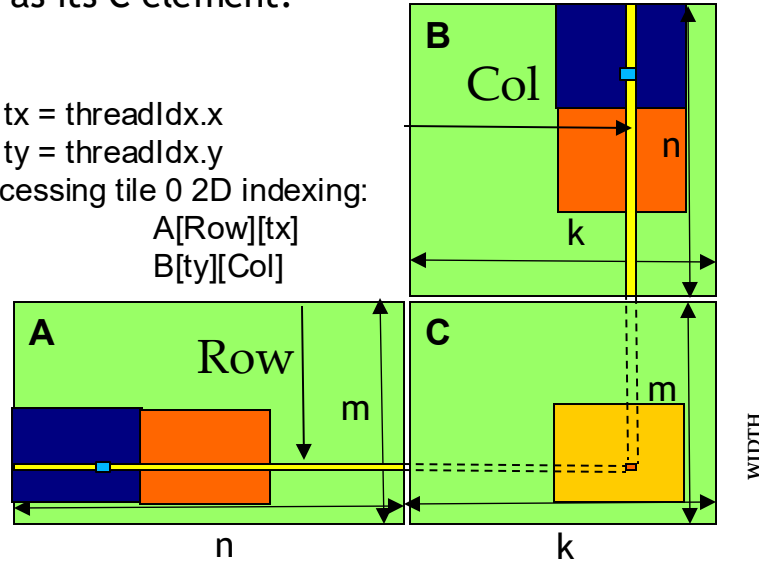
→ $N[p * \text{TILE_WIDTH} + ty][\text{Col}]$
 $N[(p * \text{TILE_WIDTH} + ty) * \text{Width} + \text{Col}]$

where **p** is the sequence number of the current phase

Tiling also optimize for coalescing

Have each thread load an A element and a B element
at the same relative position as its C element.

```
int tx = threadIdx.x  
int ty = threadIdx.y  
Accessing tile 0 2D indexing:  
A[Row][tx]  
B[ty][Col]
```

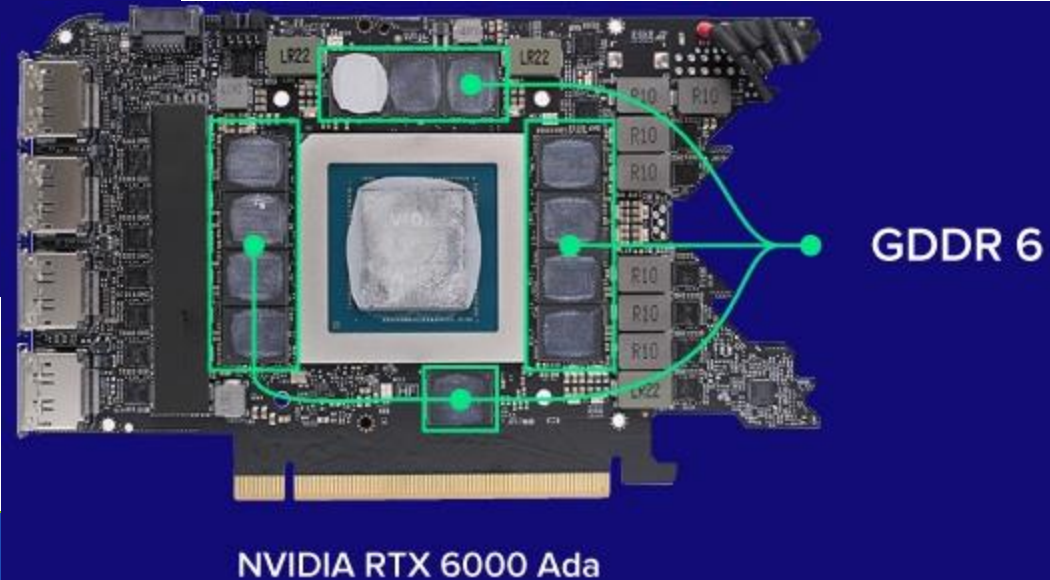
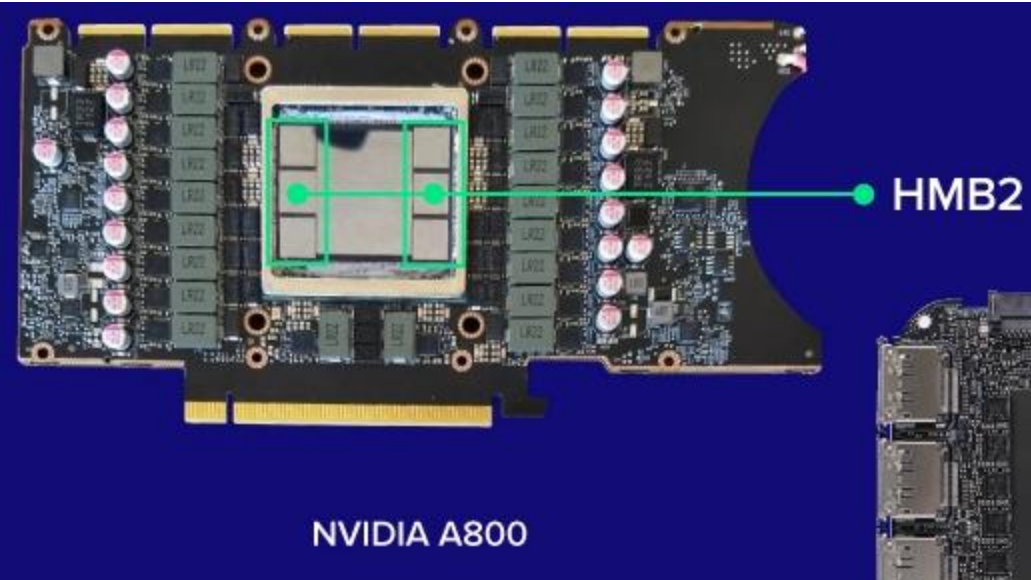


MODERN GPU MEMORY

GPU Memory Trends

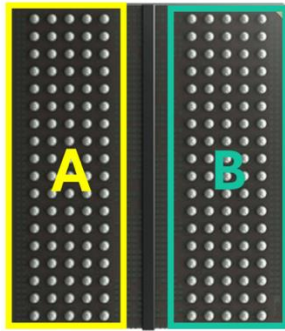
- ▶ P100 – First GPU to support HBM2. 732 GB/s.
 - ▶ GTX 10 series – GDDR5. Up to 500 GB/s.
- ▶ V100 – HBM2. 900 GB/s.
 - ▶ RTX 20 series (Turing) – GDDR6. Up to ~600 GB/s.
- ▶ A100 – HBM2. 1555 GB/s.
 - ▶ RTX 30 series – GDDR6. Up to 1 TB/s.
- ▶ H100 – HBM3. 3 TB/s.
 - ▶ RTX 40 series (Ada Lovelace) – GDDR6. Up to 1 TB/s.
- ▶ B100 – HBM3e. 8 TB/s.
 - ▶ RTX 50 series – GDDR7. Up to ~2 TB/s.

GDDR6 vs HBM2

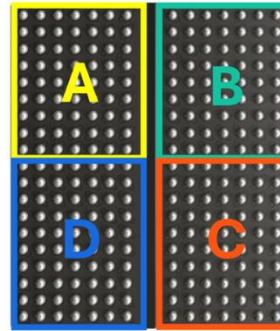


GDDR6 vs GDDR7

- › Higher data rate
- › Different signaling
- › More channels



GDDR6
Package



GDDR7
Package

Figure 3: GDDR6 Package with two channels of memory (left) vs. GDDR7 Package with four channels of memory (right)

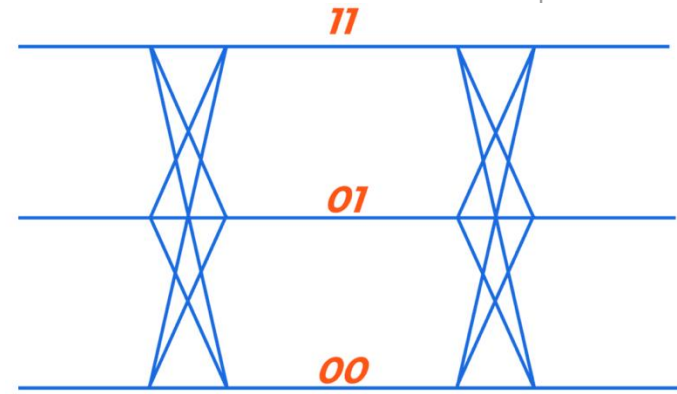


Figure 1: PAM3 Signaling

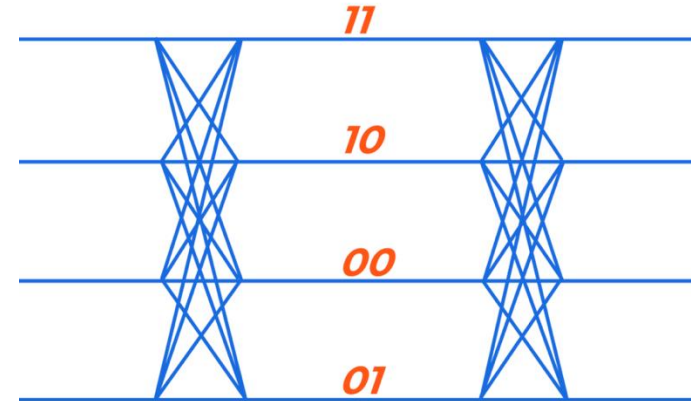
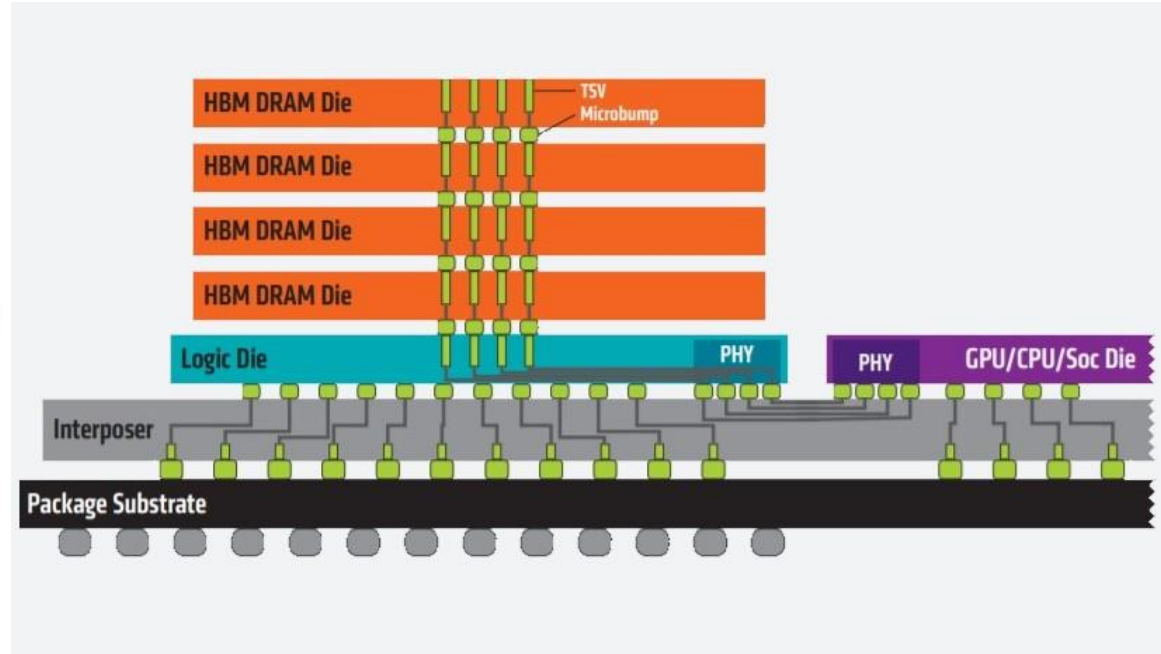
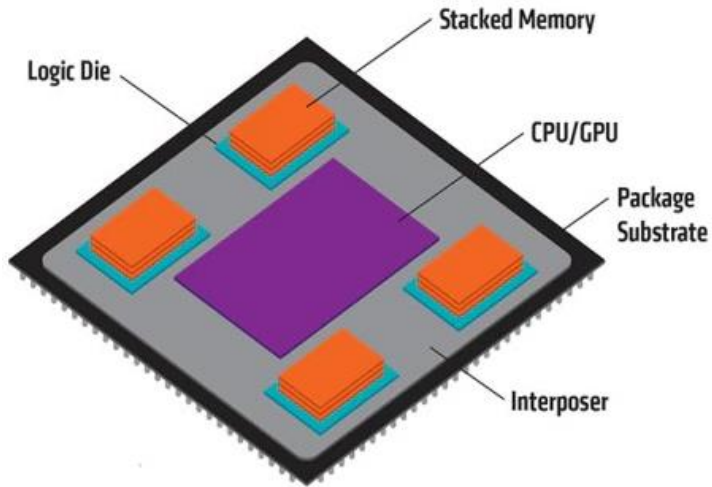


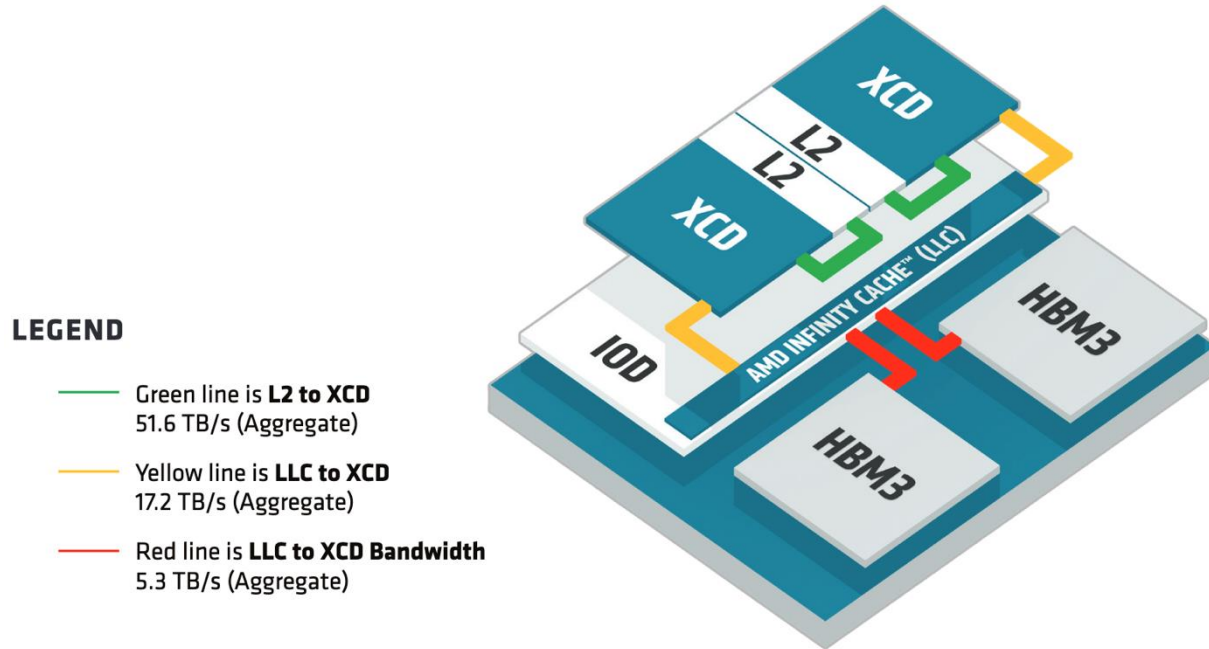
Figure 2: PAM4 Signaling

HBM2



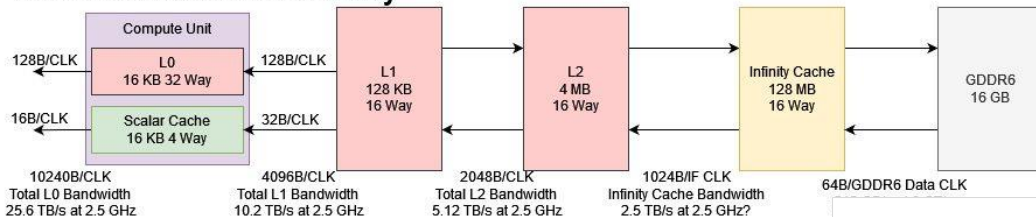
HBM3 (AMD MI300X)

Figure 6. AMD CDNA™ 3 architecture memory architecture diagram

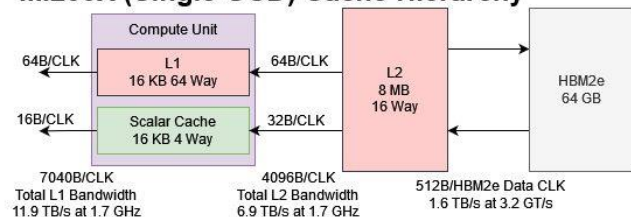


Infinity Cache

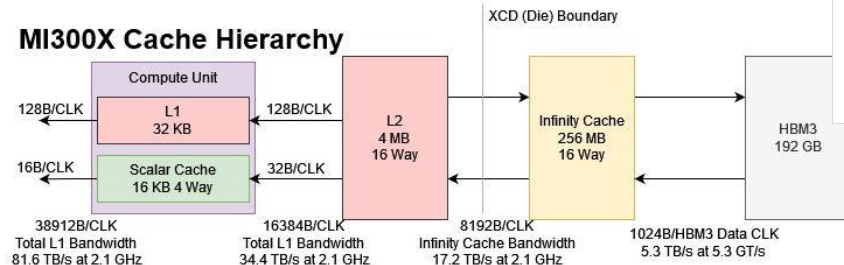
RX 6900 XT Cache Hierarchy



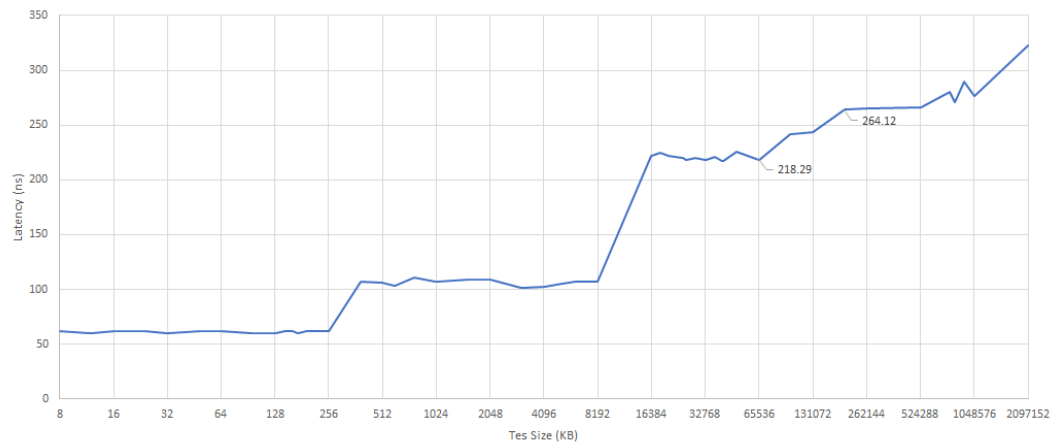
MI250X (Single GCD) Cache Hierarchy



MI300X Cache Hierarchy

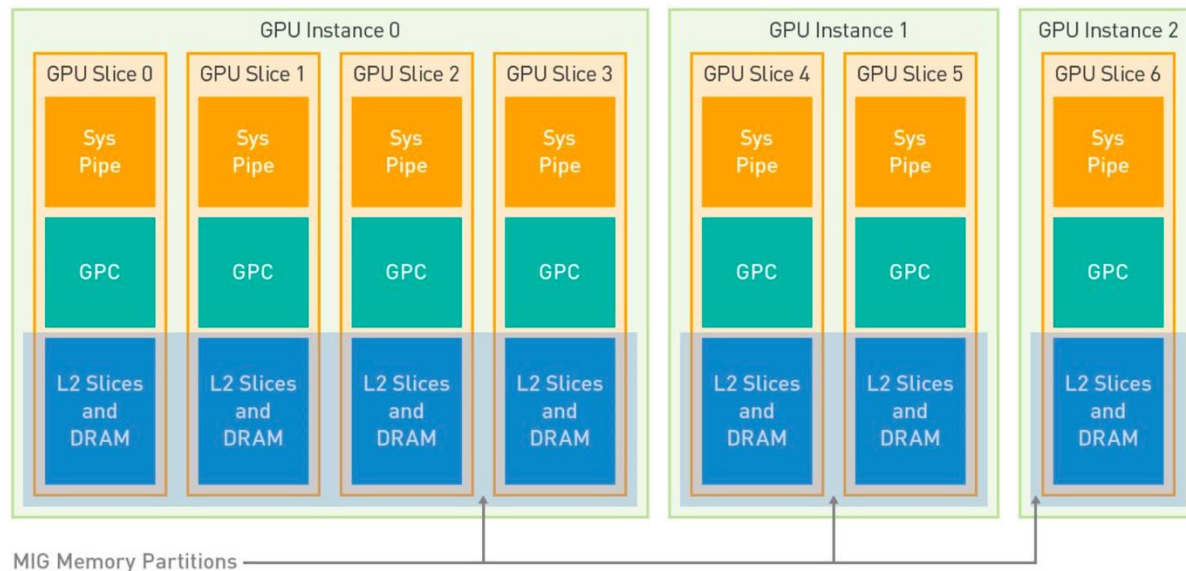


AMD Instinct MI300X, Cache and Memory Latency with 4096B Stride



More modern GPU memory considerations

- Modern GPUs can be sliced up (Nvidia MIG, AMD Partitions)



(Note that unit sizes depicted do not correspond to actual physical area on the GPU die.)

Figure 22. Example MIG compute configuration with three GPU Instances.